# 2. Opzetten van een databank

We gebruiken Postgres als databank en PGAdmin als admin interface.

We runnen alles in docker.

## 2.1 Filestructuur

Dit is vrij te kiezen, maar hieronder geef ik een voorbeeld hoe de filestructuur ineen zou kunnen zitten:

Reeds bestaand:

/GIT_PULL/
/WORKDIR/

Projects folder, op gelijk niveau:

/PROJECTS/ → de directory waar de Pycharm projects in terecht komen.
- /docker-compose.db.yml → de file waarin de PGAdmin en de Postgres server staan
- /docker-compose.work.yml → de file waarin de werkomgeving van Python staat
- /dockerfile → de dockerfile voor de werkomgeving
- /requirements.txt → de requirements van de werkomgeving

## 2.2 Docker network

Om de containers binnen de docker-compose.db te laten communiceren met elkaar, worden deze aan een docker netwerk gekoppeld:

```
networks:
  - yournetwork
```

Onderaan de yml file geven we aan dat dit een extern netwerk is – dus een bestaand docker netwerk:

```
networks:
  yournetwork:
    external: true
```

Daarom moet je dit netwerk aanmaken voor je de containers start! Dit doe je als volgt:

```
olivier@olivier-MS-7B17:~/code$ docker network create yournetwork
f4b0fc96e06b4f8a7983d2e005ca23fdde32d6d2ba56a8f635f8442ca3ba6439
```

Je kan het netwerk inspecteren en kijken wat erin zit via docker network inspect yournetwork:

```
olivier@olivier-MS-7B17:~/code$ docker network inspect yournetwork
[
    {
        "Name": "yournetwork",
        "Id": "f4b0fc96e06b4f8a7983d2e005ca23fdde32d6d2ba56a8f635f8442ca3ba6439",
        "Created": "2024-10-25T14:06:44.874295865+02:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.22.0.0/16",
                    "Gateway": "172.22.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {},
        "Labels": {}
    }
]
olivier@olivier-MS-7B17:~/code$
```

Het netwerk bestaat nu.  Nu kunnen we de databank en de admin opstarten, om uiteindelijk ook onze Pycharm omgeving te koppelen.

## 2.3 Opstart databank en admin interface

Om de databank en de admin interface op te starten ga je in de terminal naar de PROJECTS folder.

Daar geef je het volgende commando in: *docker compose -f docker-compose.db.yml up -d*

```
olivier@olivier-MS-7B17:~/code$ docker compose -f docker-compose.db.yml up -d
```

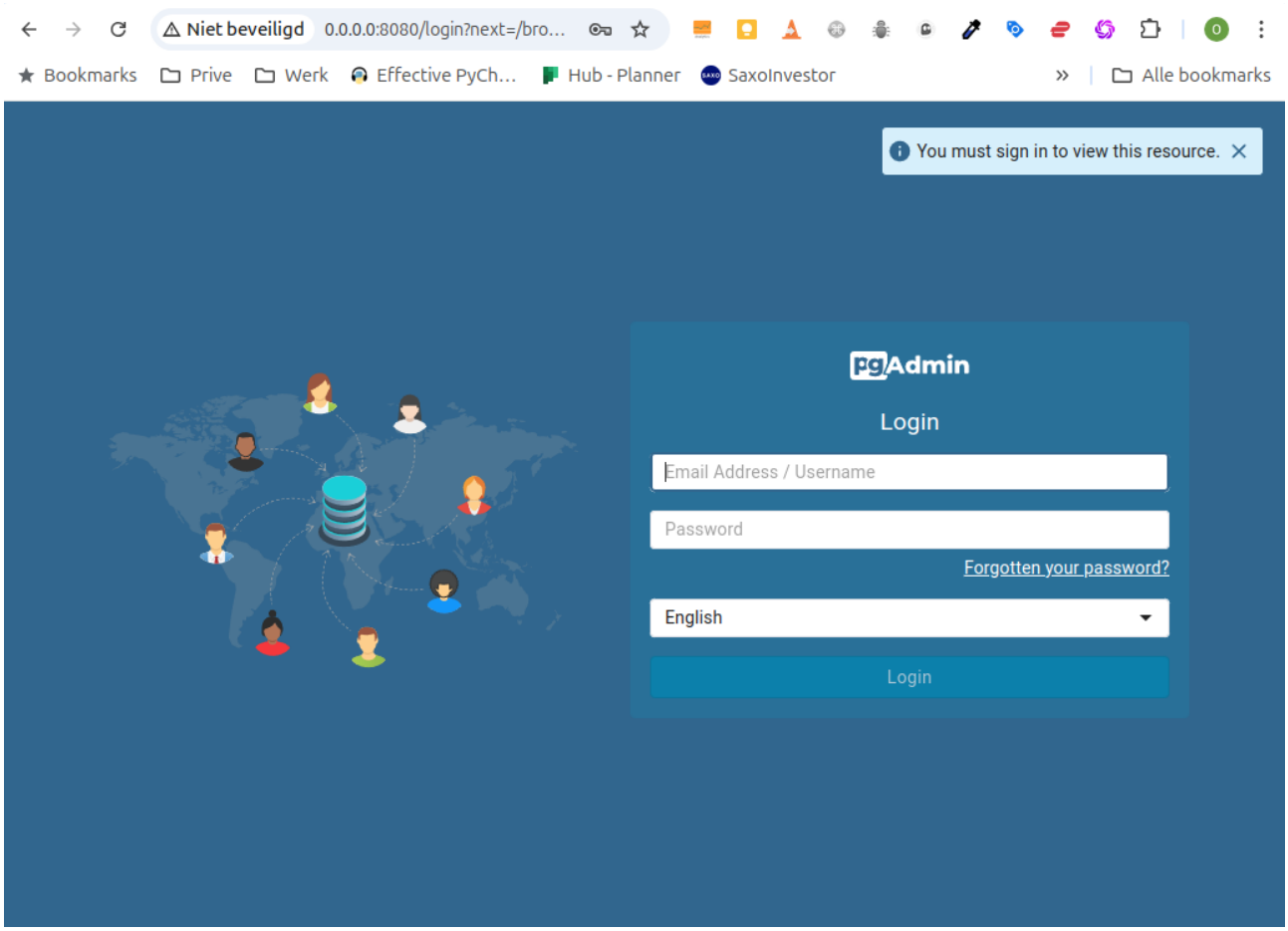
De containers runnen nu:

```
[+] Running 2/2
 ✔ Container mypostgres  Started
 ✔ Container pgadmin     Started
```

*→ Check nu even opnieuw je docker netwerk met het inspect commando. Zie je dat er nu 2 containers in vermeld worden?*

***De PGAdmin interface is nu ook live.***

Nu kan je naar 0.0.0.0:8080 surfen op je PC. Daar krijg je het inlogscherm van PGAdmin:



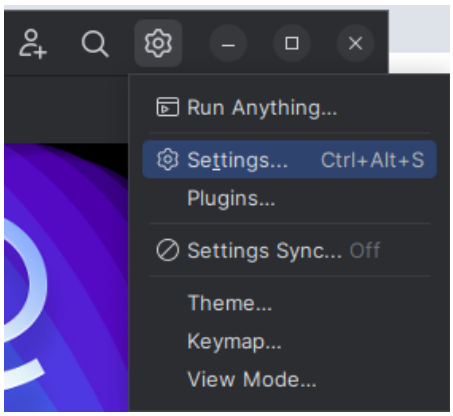

Daar kan je inloggen met de gegevens die je hebt ingegeven in je docker-compose.db.yml file bij PGADMIN_DEFAULT_EMAIL en PGADMIN_DEFAULT_PASSWORD.

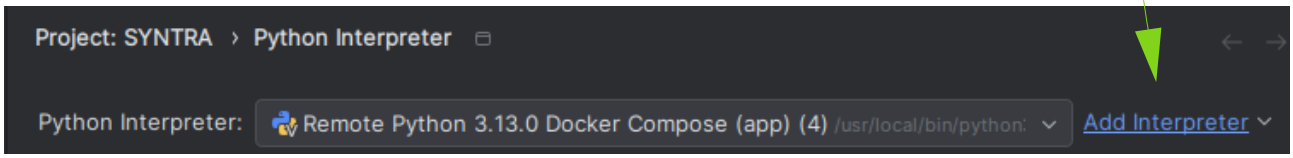
# 2.4 PyCharm toegang geven tot de databank

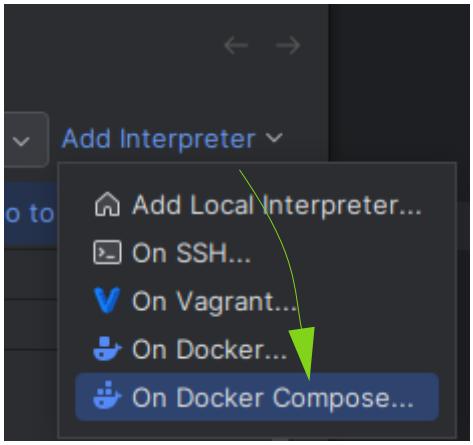
Aangezien PyCharm ook gebruik maakt van een docker container als interpreter, kan PyCharm niet zomaar de database gebruiken. We moeten de interpreter container ook aan het docker network koppelen. Dit doen we door als omgeving in PyCharm nu ook een docker compose te kiezen die dadelijk aan hetzelfde docker netwerk is gekoppeld.

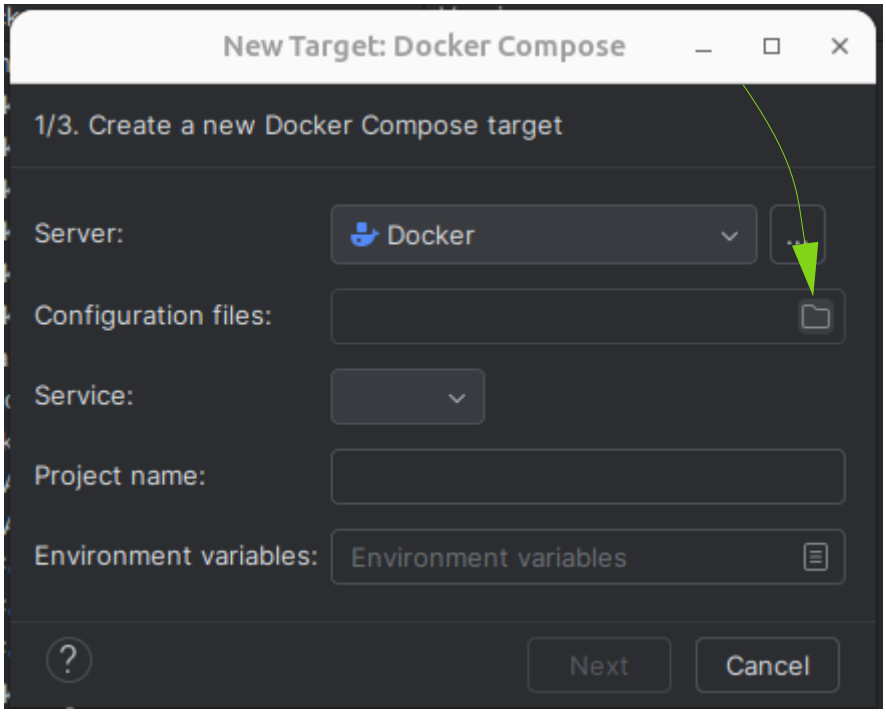
Dit is de docker-compose.work.yml

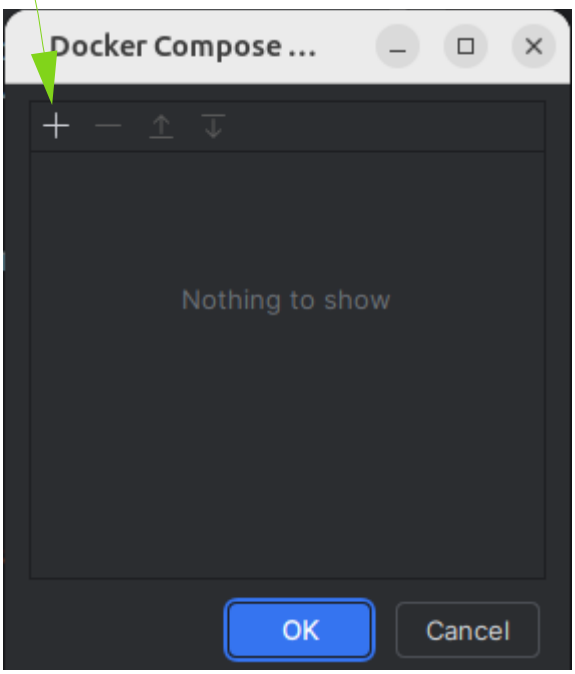
Deze starten we niet manueel op, want er is geen proces om de container draaiend te houden. Deze sluit dus dadelijk weer af. We stellen deze als interpreter in via PyCharm project settings:


Run Anything...
Settings... Ctrl+Alt+S
Plugins...
Settings Sync... Off
Theme...
Keymap...
View Mode...


Project: SYNTRA › Python Interpreter
Python Interpreter: Remote Python 3.13.0 Docker Compose (app) (4) /usr/local/bin/python
Add Interpreter


Add Interpreter
Add Local Interpreter...
On SSH...
On Vagrant...
On Docker...
On Docker Compose...


New Target: Docker Compose
1/3. Create a new Docker Compose target
Server:
Docker
Configuration files:
Service:
Project name:
Environment variables:
Environment variables
Next
Cancel




Klik op de + en selecteer de docker-compose.work.yml file.  Doorloop de daarop volgende stappen en je PyCHarm omgeving gebruikt nu de compose omgeving die is gekoppeld aan het netwerk.

→ *Check nu even opnieuw je docker netwerk met het inspect commando.  Zie je dat er nu 3 containers in vermeld worden?*